

Tarea 1: Escape de la Torre

Inteligencia Artificial

Benjamin Jimenez

Septiembre 2026

Repositorio: <https://github.com/Jhonweedmann/Tarea1-InteligenciaArtificial>

Resumen

Se simula la evacuación de un piso de una torre en llamas y se comparan cinco algoritmos de navegación: dos de búsqueda no informada (BFS, UCS), dos de búsqueda informada (A^* , Greedy Best-First) y un algoritmo genético. El entorno es dinámico: el fuego avanza de forma irreversible, las celdas tienen capacidad limitada y los agentes replanifican sus rutas. Sobre 3000 simulaciones (3 mapas, 5 algoritmos, 200 iteraciones, 80 agentes), la supervivencia queda limitada principalmente por el cuello de botella de la salida y por el tiempo que tarda el fuego en rodearla. Ninguna diferencia entre algoritmos es estadísticamente significativa tras corregir por comparaciones múltiples. Un análisis de sensibilidad (velocidad del fuego y mapas de 25×25) confirma que el resultado no se debe al tamaño de los mapas, y muestra un efecto pequeño pero consistente: Greedy es el algoritmo con menor supervivencia.

1. Introducción

El objetivo es implementar el sistema de navegación de un grupo de agentes que debe alcanzar la única salida de un piso antes de ser alcanzado por el fuego, minimizando las bajas y mitigando la congestión. Se implementan tres paradigmas: búsqueda no informada, búsqueda informada con heurística admisible y optimización bioinspirada. El desempeño se evalúa mediante un benchmark comparativo de 200 iteraciones por combinación de mapa y algoritmo.

2. Descripción del entorno

2.1. Grilla y mapas

El espacio se discretiza en una grilla regular cuyas celdas pueden ser libres, muros, fuego o salida. Cada piso tiene una única salida. Los tres mapas se diseñaron a mano para que cada uno represente la topología pedida en el enunciado (figura 1):

- **Mapa 1** (7×7 , alta densidad / cuello de botella): retícula de pasillos de ancho 1 que converge en un embudo. Una sola celda da acceso a la salida, por lo que todo el flujo pasa por ella.
- **Mapa 2** (8×8 , densidad media / laberinto corporativo): cuatro salas separadas por muros y conectadas por puertas de una celda, con tabiques que forman rincones ciegos. A la sala de la salida se entra por dos puertas.
- **Mapa 3** (9×9 , baja densidad / dispersión abierta): planta abierta con pilares sueltos y la salida al centro, accesible por sus cuatro lados.

F F	F . . # . . F	F
. # . # . # .	. # . . # . .	. # # .
.	. # . # . # .	 #
. # . # . # .	. ##### . #	
.	. . . #	. . # . E . # . .
## . . ##	. # . # . # . .	
### E ###		 #
	. . . # . . E .	. # # .
		 F
Mapa 1	Mapa 2	Mapa 3

Figura 1: Mapas del benchmark. # muro, . libre, E salida, F foco de fuego fijo.

Para que los niveles de densidad no dependan solo de la apariencia, se midieron con el script `analysis/metricas_mapas.py` (tabla 1). Las rutas independientes son el máximo de caminos sin celdas en común que llegan a la salida desde celdas a distancia ≥ 3 ; por el teorema de Menger, coincide con el mínimo de celdas que habría que bloquear para aislarla, y se calcula como un flujo máximo.

Tabla 1: Métricas topológicas de los mapas.

	Mapa 1	Mapa 2	Mapa 3
Tamaño	7×7	8×8	9×9
Densidad de muros	33 %	27 %	10 %
Celdas libres	30	44	70
Agentes por celda libre (80 agentes)	2.67	1.82	1.14
Rutas independientes hacia la salida	1	3	4
Intersecciones (celdas con ≥ 3 vecinos)	10	19	60
Callejones sin salida	0	2	0

La densidad de muros y de personas decrece del mapa 1 al 3, y el número de rutas hacia la salida crece. La densidad de muros del mapa 1 no puede ser mucho mayor: con 80 agentes y capacidad 3 por celda se necesitan al menos 27 celdas libres.

2.2. Congestión: costo y capacidad

La congestión se modela en dos niveles.

Costo de planificación. Entrar a una celda c cuesta

$$\text{costo}(c) = 1 + k \cdot \text{ocupación}(c)^2, \quad k = 0.1, \quad (1)$$

un modelo cuadrático que penaliza fuertemente las celdas con varios agentes. UCS, A* y el algoritmo genético usan este costo; BFS y Greedy no.

Capacidad física. Cada celda admite como máximo 3 agentes simultáneos. Si la celda siguiente de la ruta está llena, el agente espera en su lugar. En la salida, los agentes evacuados se retiran al final del turno, por lo que evacúan como máximo 3 agentes por turno. Esto reproduce el embotellamiento que retrasa el flujo.

2.3. Propagación del fuego

Cada mapa tiene dos focos fijos en esquinas alejadas de la salida. Además, en cada iteración se enciende al azar cerca del 5% de las celdas libres según la semilla, cuidando que la salida siga conectada. Cada $k = 3$ turnos el fuego se expande a las celdas ortogonales adyacentes. Las celdas quemadas quedan intransitables para el resto de la simulación y la salida está protegida. Un agente alcanzado por el fuego es una baja.

2.4. Movimiento y replanificación

Los agentes solo se desplazan en forma ortogonal (arriba, abajo, izquierda, derecha) o esperan. Cualquier paso que no sea a una celda vecina transitable se rechaza. Cada vez que el fuego se propaga, todos los agentes vivos recalculan su ruta con el algoritmo en evaluación, considerando la ocupación actual.

2.5. Orden de un turno

1. Cada agente vivo intenta avanzar un paso de su ruta, o espera si la celda está llena.
2. Los agentes que llegaron a la salida se retiran del entorno.
3. El fuego avanza (cada 3 turnos) y los agentes en celdas quemadas mueren.
4. Si el fuego avanzó, los agentes replanifican.

3. Algoritmos implementados

3.1. Búsqueda no informada

BFS (Breadth-First Search). Expande nodos por niveles con una cola FIFO y un conjunto de visitados. Encuentra la ruta con menos pasos, pero ignora la congestión. Sirve como línea base.

UCS (Uniform Cost Search). Expande el nodo de menor costo acumulado con una cola de prioridad. Como el costo de cada celda incluye la congestión, encuentra la ruta de menor costo total según la ocupación al momento de planificar.

3.2. Búsqueda informada

A*. Ordena la frontera por $f(n) = g(n) + h(n)$, donde $g(n)$ es el costo acumulado con congestión y $h(n)$ la distancia Manhattan a la salida:

$$h(n) = |r_n - r_s| + |c_n - c_s|. \quad (2)$$

La heurística es admisible: con movimiento ortogonal cada paso cuesta al menos 1, así que nunca sobreestima el costo real. Por lo tanto, A* es óptimo respecto del costo.

Greedy Best-First. Ordena la frontera solo por $h(n)$, sin costo acumulado. Explora menos nodos, pero no es óptimo y no considera la congestión.

3.3. Optimización bioinspirada: algoritmo genético

- **Individuo:** secuencia de acciones (arriba, abajo, izquierda, derecha, esperar). Se decodifica desde la posición del agente; una acción que choca con un muro, el fuego o el borde se interpreta como esperar. Así, toda ruta generada respeta el movimiento ortogonal de una casilla.

- **Población inicial:** 15 individuos; 60 % sembrados con la ruta de A^* (convertida a acciones, más una cola aleatoria de hasta 5 acciones) y 40 % aleatorios, con longitud entre 20 y 150.

- **Fitness:**

$$F = \begin{cases} 200 - \text{pasos} - 0.05 \cdot \text{costo} & \text{si la ruta llega a la salida,} \\ -d_{\text{Manhattan}} - 2 \cdot \text{costo} & \text{si no llega.} \end{cases} \quad (3)$$

- **Selección:** torneo de tamaño 3.
- **Cruce:** un punto de corte.
- **Mutación:** cada gen se reemplaza por una acción aleatoria con probabilidad 0.2.
- **Elitismo:** se conserva el 20 % mejor de cada generación.
- **Generaciones:** 20. Se ejecuta al inicio y en cada replanificación.

4. Resultados experimentales

4.1. Configuración del benchmark

- 3 mapas $\times$ 5 algoritmos $\times$ 200 iteraciones = 3000 simulaciones, con 80 agentes cada una.
- Cada iteración usa una semilla aleatoria distinta, que determina el fuego inicial y las posiciones de los agentes. Los agentes se ubican al azar solo en celdas libres que aún tienen capacidad.
- $k = 0.1$ (congestión), capacidad 3 por celda, propagación cada 3 turnos, máximo 500 turnos.

Métricas.

- **Tasa de supervivencia:** $N_{\text{sobrevivientes}}/N_{\text{total}}$ por simulación, promediada sobre las 200 iteraciones. Se reporta el intervalo de confianza del 95 % de la media, $\bar{x} \pm 1.96 \sigma / \sqrt{n}$.
- **Tiempo de despeje:** turno en que el último sobreviviente alcanza la salida. Se reportan media, desviación estándar, mínimo y máximo.

4.2. Tabla de resultados

Tabla 2: Resultados del benchmark (200 iteraciones por fila, 80 agentes). En negrita, la mayor supervivencia de cada mapa.

Mapa	Algoritmo	Supervivencia (IC 95 %)	Media	Std	Mín	Máy
map1	BFS	0.397 [0.367, 0.427]	11.87	6.25	2	21
map1	UCS	0.394 [0.365, 0.423]	12.13	6.17	3	21
map1	A*	0.402 [0.374, 0.429]	12.33	5.94	2	21
map1	Greedy	0.408 [0.382, 0.434]	12.62	5.37	3	20
map1	Genético	0.419 [0.391, 0.446]	12.86	5.89	3	21
map2	BFS	0.454 [0.426, 0.482]	13.28	5.38	3	24
map2	UCS	0.426 [0.396, 0.457]	12.60	5.73	2	24
map2	A*	0.417 [0.390, 0.443]	12.44	5.05	3	25
map2	Greedy	0.402 [0.374, 0.430]	13.02	5.79	3	25
map2	Genético	0.459 [0.430, 0.489]	13.62	5.54	3	25
map3	BFS	0.451 [0.431, 0.470]	12.55	3.78	5	21
map3	UCS	0.448 [0.426, 0.469]	12.36	4.15	3	21
map3	A*	0.442 [0.421, 0.464]	12.23	4.10	3	21
map3	Greedy	0.420 [0.400, 0.440]	11.70	3.92	3	21
map3	Genético	0.439 [0.417, 0.462]	12.15	4.38	3	21

Media, Std, Mín y Máx corresponden al tiempo de despeje en turnos. En las 3000 simulaciones hubo al menos un sobreviviente.

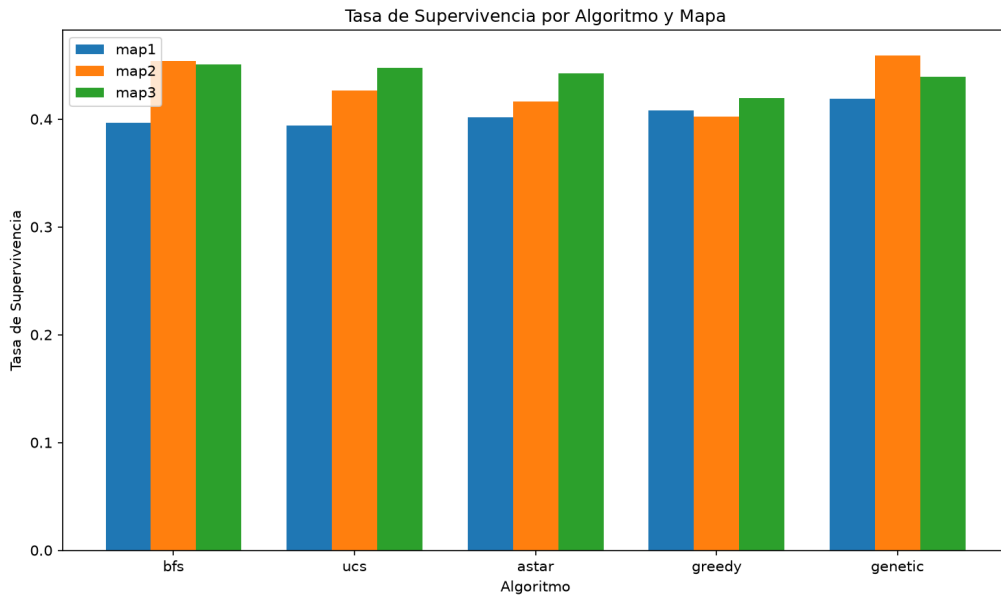


Figura 2: Tasa de supervivencia media por algoritmo y mapa.

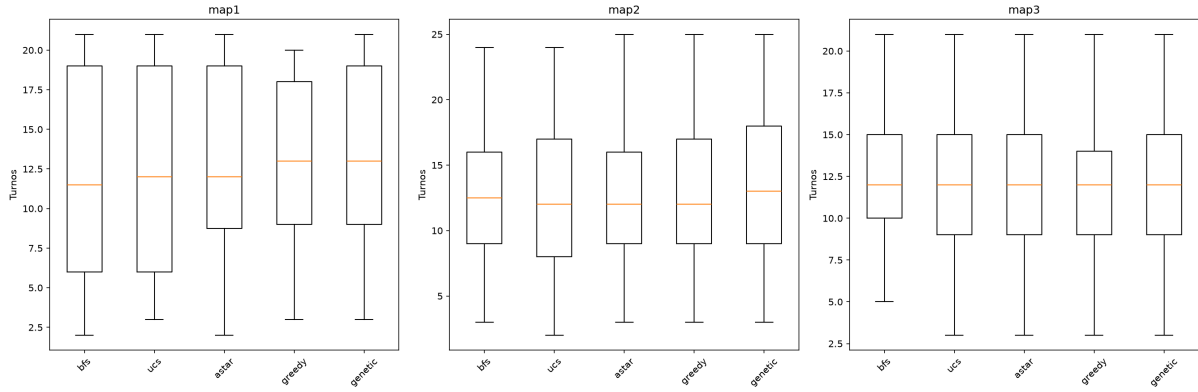


Figura 3: Distribución del tiempo de despeje (turnos) por algoritmo en cada mapa.

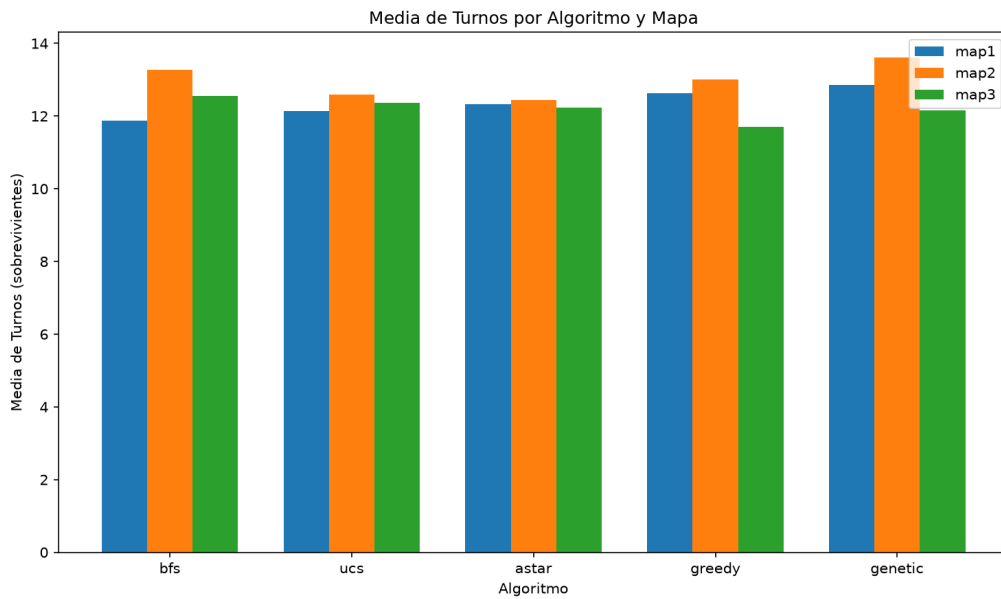


Figura 4: Tiempo de despeje medio por algoritmo y mapa.

4.3. Pruebas de significancia

Para cada mapa se compararon las 10 parejas de algoritmos con una prueba z de diferencia de medias (varianzas distintas, $n = 200$ por grupo) y se corrigió por comparaciones múltiples con el método de Holm. **Ninguna diferencia es significativa al 5% tras la corrección.** Sin corregir, cuatro parejas tienen $p < 0.05$:

Tabla 3: Parejas con $p < 0.05$ antes de corregir (supervivencia).

Mapa	Comparación	Diferencia	p	p (Holm)
map2	Greedy vs. Genético	-0.057	0.006	0.056
map2	BFS vs. Greedy	+0.052	0.011	0.095
map2	A* vs. Genético	-0.043	0.034	0.271
map3	BFS vs. Greedy	+0.031	0.030	0.303

Tres de las cuatro involucran a Greedy como el peor algoritmo. El análisis de sensibilidad (sección 4.7) muestra que ese patrón se repite en todas las configuraciones.

4.4. Análisis: el cuello de botella de la salida

El resultado central es que la supervivencia queda limitada principalmente por el entorno y no por el algoritmo. Dos mecanismos lo explican; ambos se midieron con el script `metrics_mapas.py` sobre las semillas del benchmark.

Capacidad de la salida. Como solo evacúan 3 agentes por turno, el número de sobrevivientes no puede superar $3T$, donde T es el tiempo de despeje. En promedio, los agentes evacuados alcanzan el 86 % (mapa 1), 87 % (mapa 2) y 96 % (mapa 3) de esa cota: la salida trabaja casi siempre a plena capacidad.

Sellado de la salida por el fuego. Aunque la celda de salida está protegida, el fuego termina ocupando todas sus celdas vecinas. Esto ocurrió en el 100 % de las semillas, en promedio en el turno 13.9 (mapa 1), 16.5 (mapa 2) y 12.8 (mapa 3). Por eso el máximo de turnos es prácticamente idéntico para todos los algoritmos (21, 24–25 y 21).

Con la salida saturada y un tiempo disponible fijado por el fuego, cualquier algoritmo que lleve agentes a la salida de forma continua obtiene una supervivencia similar: evacuar a los 80 agentes requiere al menos $80/3 \approx 27$ turnos, y el fuego sella la salida antes.

4.5. Análisis por algoritmo

- **Greedy** tiene la menor supervivencia en los mapas 2 y 3. Ordena la frontera solo por distancia Manhattan, que en un laberinto es engañosa: lleva a los agentes hacia muros y rincones ciegos que están cerca de la salida en línea recta, y la ruta que encuentra puede ser más larga que la óptima. El efecto es pequeño (2 a 6 puntos), pero es el único que se repite en todos los experimentos.
- **BFS** es igual o mejor que UCS y A* en los mapas 1 y 2. La congestión que ven UCS y A* es la del instante de planificación: desvían a los agentes por rutas más largas para evitar celdas ocupadas, pero la ocupación cambia en cuanto los agentes se mueven, y la capacidad física de las celdas ya regula el flujo. El desvío cuesta pasos y no ahorra espera.
- **UCS y A*** obtienen resultados prácticamente idénticos. Usan la misma función de costo y ambos son óptimos, así que encuentran rutas de igual costo; A* solo expande menos nodos gracias a la heurística. Sus diferencias se deben a desempates entre rutas del mismo costo.
- **El algoritmo genético** tiene la mayor supervivencia media en los mapas 1 y 2, aunque sin significancia estadística. Parte de rutas de A* y la mutación agrega esperas y variaciones pequeñas; una posible explicación, no verificada, es que esas variaciones reparten a los agentes en el tiempo y reducen los bloqueos. Es el más costoso: concentra la mayor parte de los 18 minutos que tomó el benchmark completo.

4.6. Análisis por mapa

- **Mapa 1** (cuello de botella): supervivencia de 0.39 a 0.42 para todos. Toda la evacuación pasa por una sola celda, así que la ruta elegida importa poco: el embudo impone el ritmo.
- **Mapa 2** (laberinto): es el mapa con más diferencia entre algoritmos (0.40 a 0.46). Con tres accesos a la salida y rincones ciegos, la elección de ruta sí influye, y es donde Greedy más se equivoca. También es donde el fuego tarda más en sellar la salida (turno 16.5), porque la sala de la salida está protegida por muros.

- **Mapa 3** (abierto): supervivencia de 0.42 a 0.45. La salida recibe agentes por sus cuatro lados y trabaja al 96 % de su capacidad, pero al estar al centro el fuego la alcanza antes (turno 12.8).

4.7. Análisis de sensibilidad

Para comprobar si la falta de diferencias se debe a que los mapas son pequeños o a que el fuego es muy rápido, se corrieron dos experimentos adicionales con el script `sensibilidad.py`. Se usaron 100 semillas, las mismas para todos los algoritmos (comparación pareada). El algoritmo genético se excluyó por su costo computacional. Los resultados se guardan en `sensibilidad.csv`, dentro de `benchmark/results/`.

Experimento A: velocidad del fuego. Mapas del benchmark con el fuego propagándose cada 3, 5 y 7 turnos.

Tabla 4: Supervivencia media según el intervalo de propagación del fuego (100 semillas pareadas; IC 95 % entre ± 0.03 y ± 0.05).

Intervalo	Mapa	BFS	UCS	A*	Greedy
3 turnos	map1	0.404	0.387	0.387	0.371
	map2	0.488	0.479	0.480	0.442
	map3	0.429	0.438	0.438	0.428
5 turnos	map1	0.578	0.557	0.557	0.533
	map2	0.600	0.584	0.584	0.563
	map3	0.643	0.650	0.650	0.634
7 turnos	map1	0.680	0.662	0.662	0.643
	map2	0.663	0.645	0.645	0.630
	map3	0.778	0.782	0.782	0.768

Un fuego más lento sube la supervivencia de todos los algoritmos por igual (de ~ 0.4 a ~ 0.7), pero la distancia entre ellos no crece: se mantiene entre 1 y 5 puntos. Aun así, el orden es muy estable: **Greedy tiene la menor supervivencia en las 9 configuraciones**, BFS supera a UCS y A* en los mapas 1 y 2, y UCS y A* coinciden.

Experimento B: tamaño del mapa. Se construyeron versiones de 25×25 de las tres topologías (retícula de pasillos con embudo, 16 salas con puertas y planta abierta con pilares), con dos focos fijos en esquinas y 80 agentes.

Tabla 5: Supervivencia media en mapas de 25×25 (100 semillas pareadas).

Fuego inicial	Mapa	BFS	UCS	A*	Greedy
Con 5 % aleatorio	map1	0.239	0.236	0.236	0.230
	map2	0.116	0.112	0.112	0.104
	map3	0.244	0.256	0.256	0.241
Solo focos fijos	map1	1.000	1.000	1.000	1.000
	map2	0.998	0.999	1.000	0.967
	map3	1.000	1.000	1.000	1.000

Agrandar el mapa no separa a los algoritmos. Con el fuego aleatorio del 5 %, el número de focos crece con el mapa y la supervivencia cae al 10–26 % para todos. Sin fuego aleatorio, el

mapa grande es demasiado fácil y casi todos se salvan. La única diferencia significativa aparece en el laberinto sin fuego aleatorio: Greedy pierde al 3.3 % de los agentes, mientras que los demás no pierden casi ninguno (IC 95 % de ± 0.006 frente a ± 0.001).

Conclusión del análisis. La similitud entre algoritmos no es un efecto del tamaño de los mapas ni de la velocidad del fuego. Es una propiedad del problema tal como está modelado: todos los algoritmos encuentran rutas de largo parecido hacia una única salida, y la supervivencia la deciden el ritmo de evacuación de la salida y el avance del fuego. El único efecto consistente es la desventaja de Greedy, que se hace visible en laberintos, donde la distancia Manhattan es una mala guía.

5. Conclusiones

- Con 80 agentes y una salida de capacidad 3, el cuello de botella común domina el resultado: la supervivencia depende sobre todo de cuánto tarda el fuego en sellar la salida.
- En el benchmark, ninguna diferencia entre algoritmos es estadísticamente significativa tras corregir por comparaciones múltiples.
- Greedy es el algoritmo más débil: tiene la menor supervivencia en todas las configuraciones del análisis de sensibilidad, y el efecto es mayor en el laberinto.
- Considerar la congestión al planificar (UCS, A*) no mejora a BFS en este modelo, porque la ocupación que se usa para planificar deja de ser válida en cuanto los agentes se mueven.
- El algoritmo genético es competitivo, con la mejor media en dos mapas, pero sin diferencia significativa y con un costo computacional mucho mayor.
- Agrandar los mapas o hacer más lento el fuego no separa a los algoritmos. Para diferenciarlos habría que cambiar el modelo: por ejemplo, un costo de congestión que anticipe la ocupación futura o varias salidas entre las que elegir.

5.1. Limitaciones

- La congestión que ven los algoritmos es la del instante de planificación; los agentes no se coordinan ni reservan celdas.
- Los agentes esperan solo cuando la celda siguiente está llena; la acción de esperar no forma parte del espacio de búsqueda de BFS, UCS, A* ni Greedy (sí del algoritmo genético).
- En el benchmark, cada algoritmo se evaluó con semillas aleatorias independientes. El análisis de sensibilidad sí usa semillas pareadas, lo que lo hace más sensible a diferencias pequeñas.
- En cerca del 3.5 % de las semillas del mapa 1 (7 de 200 en una prueba), el fuego aleatorio deja menos de 27 celdas libres y algunas celdas empiezan con más de 3 agentes.
- El análisis de sensibilidad excluye al algoritmo genético por su costo computacional.

5.2. Trabajo futuro

- Planificación cooperativa con reserva de celdas en el tiempo (por ejemplo, Cooperative A*).
- Incluir la acción de esperar y la predicción del avance del fuego en la búsqueda.
- Heurísticas que consideren la distancia al frente de fuego.

- Usar semillas pareadas en el benchmark principal.
- Visualización gráfica del entorno multiagente.

6. Fuentes y uso de IA generativa

- Russell, S. y Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4.^a ed.). Pearson. — BFS, UCS, A*, Greedy Best-First.
- Goldberg, D. E. (1989). *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley. — Algoritmo genético.
- La heurística Manhattan es un concepto estándar de inteligencia artificial.
- No se utilizaron repositorios externos para los algoritmos de búsqueda.

Uso de IA generativa. Se utilizó Claude Code (Anthropic) como asistente para corregir errores del entorno y la simulación (capacidad por celda, conteo de ocupación, métrica de tiempo de despeje), para reescribir la representación del algoritmo genético como secuencia de acciones, para rediseñar los tres mapas según las topologías del enunciado, para implementar los scripts de métricas de mapas y de análisis de sensibilidad, y para redactar este informe a partir de los resultados.